

《计算机组成原理》实验报告

年级、专业、班级		2024 级计算机科技, 01 班, 03 班, 02 班		姓名	BROEKHOFROBIN, 严浩睿, 吴林兴
实验题目	实验三 简易单周期 CPU 实验				
实验时间	4 月, 26 号	实验地点	DS1421		
实验成绩		实验性质	☐ 验证性 ☒ 设计性 ☐ 综合性		
教师评价: ☐算法/实验过程正确; ☐源程序/实验内容提交 ☐程序结构/实验步骤合理; ☐实验结果正确; ☐语法、语义正确; ☐报告规范; 其他: 评价教师签名:					
一、实验目的 (1) 掌握不同类型指令在数据通路中的执行路径。 (2) 掌握 Vivado 仿真方式。					
二、实验项目内容 (1) Datapath, 其中主要包含 alu(实验一已完成), PC(实验二已完成), adder、mux2、signext、sl2(其中 adder、mux2 数字逻辑课程已实现, signext、sl2 参见实验原理), (2) Controlller(实验二已完成), 其中包含两部分, 分别为 main_decoder, alu_decoder。 (3) 指令存储器 inst_mem(Single Port Ram), 数据存储器 data_mem(Single Port Ram); 使用 Block Memory Generator IP 构造指令, 注意考虑 PC 地址位数统一。(参考实验二) (4) 参照实验原理, 将上述模块依指令执行顺序连接。实验给出 top 文件, 需兼容 top 文件端口设定。 (5) 实验给出仿真程序, 最终以仿真输出结果判断是否成功实现要求指令。					

报告创建时间:

三、实验过程或算法（源程序）

模块结构与解释

号	模块名称	描述
1.	Top. v	把所有模块联合起来，要定义 clk 与 rst
2.	--miptest. v	本实验提供的 mips 指令文件
3.	--blk_gen_mem_0	数据 RAM
4.	--mips. v	为了连接 datapath 与 controller 模块。生成所有控制信号
5.	----controller. v	控制器模块，要跟据
6.	-----main_decoder. v	要根据指令的操作码 inst[31:26] 决定要运行哪一类型指令，与不同的指令也有不同的控制信号需求
7.	-----alu_decoder. v	处理 alu 部件需要运行哪个运算（比如：add, sub, or, and）
8.	----datapath. v	连接所有处理器的部件
9.	-----program_counter. v	每个周期要运行下个指令，于是定义在下个周期 pc 指令地址要加 4
10.	-----adder. v	32 位的加法器，应用包括 pc 与 branch 位置运算
11.	-----signext. v	能把 16 位立即数成为 32 位
12.	-----s12. v	左移两位，用于在分支与跳转指令
13.	-----mux2. v	2: 1 多路选择器，控制器要在

		不同的信号使用它
14.	-----regfile.v	本实验提供的寄存器文件，一共有 32 个寄存器
15.	-----alu.v	处理器的运算部件

控制器接口定义（controller 模块的信号）

信号名	方向	位宽	功能描述
instr	输入	6	指令的操作码，来自 instr[31:26]。操作码决定要执行哪个指令类型
instr_valid	输入	1	因为 ram 有延时，需要检查当前的指令为有效的指令
jump	输出	1	若信号为 1，要跳转到新 pc 地址
branch	输出	1	若信号为 1，要运行 beq 指令
alu_src	输出	1	用于在有立即数运算
mem_write	输出	1	写存储（用于在 sw 指令）
mem_to_reg	输出	1	存储到寄存器（用于在 lw 指令）
reg_write	输出	1	写到寄存器
reg_dst	输出	1	写到寄存器的对象
func	输出	6	R 型指令辅助操作码
alu_control	输出	3	控制 alu 的运算（包括 add, sub, or, and）
alu_op	输出	2	Main_decoder 与 alu_decoder 之间的 alu 控制信号

Datapath 接口定义（datapath 模块的信号）

信号名	方向	位宽	功能描述
clk	输入	1	时钟信号
rst	输入	1	复位信号，若 rst 为高将 pc 设置为 0
mem_to_reg	输入	1	存储到寄存器（用于在 lw 指令）
pc_src	输入	1	PC 源选择：0=PC+4，1=分支地址
alu_src	输入	1	ALU 输入 B 源选择：0=寄存器堆读出数据，1=立即数
reg_dst	输入	1	写到寄存器的对象
reg_write	输入	1	寄存器堆写使能
jump	输入	1	若信号为 1，要跳转到新 pc 地址
alu_control	输入	3	控制 alu 的运算（包括 add, sub, or, and）
overflow	输出	1	若 alu 结果有溢出，将 overflow 设置为 1
zero	输出	1	Alu 运算结构为 0，将 zero 设置为 1
pc	输出	32	当前指令地址
instr	输入	32	Mips 指令
alu_out	输出	32	Alu 的运算结构
write_data	输出	32	要写入存储器的数据
read_data	输入	32	从数据存储器读出的数据

指令分析

R 型

31-26	25-21	20-16	15-11	10-6	5-0
操作码	rs	rt	rd	shamt	funct

I 型

31-26	25-21	20-16	15-0
操作码	rs	rt	立即数

J 型

31-26	25-0
操作码	对象地址

```

shiyanke3 > coe > ASM mipstest.asm
1  .data
2  |   a: .word 3
3  .text 0x0000
4  main:
5      addi $2, $0, 5      # initialize $2=5      0  20020005
6      addi $3, $0, 12     # initialize $3=12     4  2003000c
7      addi $7, $3, -9     # initialize $7=3      8  2067fff7
8      or $4, $7, $2       # $4 <= 3 or 5 = 7     c  00e22025
9      and $5, $3, $4      # $5 <= 12 and 7 = 4   10 00642824
10     add $5, $5, $4       # $5 = 4 + 7 = 11     14 00a42820
11     beq $5, $7, end      # shouldn't be taken   18 10a7000c
12     slt $4, $3, $4      # $4 = 12 < 7 = 0     1c 0064202a
13     beq $4, $0, around  # should be taken     20 10800002
14     nop                 # empty inst          24 00000000
15     addi $5, $0, 0      # shouldn't happen    28 20050000
16 around:
17     slt $4, $7, $2      # $4 = 3 < 5 = 1      2c 00e2202a
18     add $7, $4, $5      # $7 = 1 + 11 = 12    30 00853820
19     sub $7, $7, $2      # $7 = 12 - 5 = 7    34 00e23822
20     sw $7, 68($3)       # [80] = 7      38 ac670044
21     lw $2, 80($0)       # $2 = [80] = 7    3c 8c020050
22     j end               # should be taken    40 08000013
23     nop                 # empty inst          44 00000000
24     addi $2, $0, 1      # shouldn't happen 48 20020001
25 end:
26     sw $2, 84($0)       # write adr 84=7   4c ac020054

```

图 1：本实验提供的 RAM 指令文件

四、实验结果及分析和（或）源程序调试过程

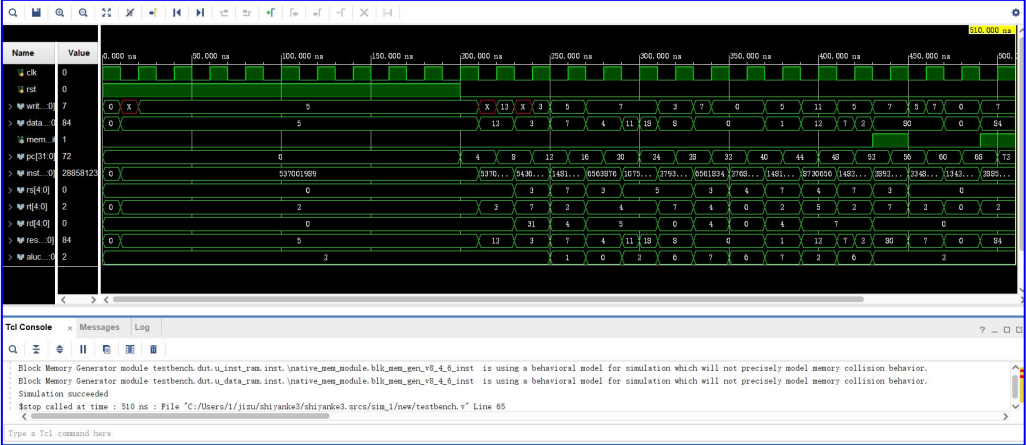


图 2：控制台写“仿真成功”。上面仿真图使用十进制来表示数据，由于 blockmemory 的局限导致有延时

1.top.v

```
module top(
    input wire clk,
    input wire rst,
    output wire[31:0] writedata,    //写数据
    output wire[31:0] dataadr,      //数据地址
    output wire memwrite           //内存写使能
);
    wire [31:0] pc;
    wire [31:0] instr;             //指令存储器读出的指令
    wire [31:0] readdata;          //从数据存储器读出的数据

    //连接处理器、指令存储器和数据存储器
    mips u_mips (
        .clk      (clk),
        .rst      (rst),
        .pc       (pc),
        .instr     (instr),
        .memwrite  (memwrite),
        .aluout    (dataadr),
        .writedata (writedata),
        .readdata  (readdata)
    );
    //实验提供的 mips 指令文件
    mipstest u_inst_ram (
        .clka  (~clk),
        .ena   (1'b1),
        .wea   (1'b0),
        .addra (pc[7:2]),
        .dina  (32'b0),
        .douta (instr)
    );
    //数据存储器
    blk_mem_gen_0 u_data_ram (
        .clka  (clk),
        .ena   (1'b1),
        .wea   ({memwrite}),
        .addra (dataadr[7:2]),
        .dina  (writedata),
        .douta (readdata)
    );
endmodule
```

4. Mips.v

```
module mips(  
    input wire clk,  
    input wire rst,  
    output wire [31:0] pc,  
    input wire [31:0] instr,  
    output wire memwrite,  
    output wire [31:0] aluout,  
    output wire [31:0] writedata,  
    input wire [31:0] readdata  
);  
  
wire memtoreg;  
wire alusrc;  
wire regdst;  
wire regwrite;  
wire jump;  
wire branch;  
wire pcsrc;  
wire zero;  
wire overflow;  
wire [2:0] alucontrol;  
  
assign pcsrc = branch & zero; //分支指令且ALU结果为0时, pcsrc为1, pc选择分支地址  
//控制器和数据通路  
controller u_controller (  
    .instr      (instr[31:26]),  
    .jump       (jump),  
    .branch     (branch),  
    .alu_src     (alusrc),  
    .mem_write  (memwrite),  
    .mem_to_reg (memtoreg),  
    .reg_write  (regwrite),  
    .reg_dst    (regdst),  
    .func       (instr[5:0]),  
    .alu_control (alucontrol)  
);  
//数据通路  
datapath u_datapath (  
    .clk      (clk),  
    .rst      (rst),  
    .mem_to_reg (memtoreg),  
    .pc_src    (pcsrc),  
    .alu_src   (alusrc),  
    .reg_dst   (regdst),  
    .reg_write (regwrite),  
    .jump      (jump),
```



```

        .alu_control (alucontrol),
        .overflow    (overflow),
        .zero        (zero),
        .pc          (pc),
        .instr       (instr),
        .alu_out     (aluout),
        .write_data  (writedata),
        .read_data   (readdata)
    );
endmodule

```

5. controller.v

```

module controller (
    //main_decoder 的信号
    input [5:0] instr,          // instruct[31:26] = 操作码

    //控制信号
    output wire jump,
    output wire branch,
    output wire alu_src,
    output wire mem_write,
    output wire mem_to_reg,
    output wire reg_write,
    output wire reg_dst,

    //alu_decoder 的信号
    input [5:0] func,          // r 型指令的辅助操作码信号
    output wire [2:0] alu_control
);
    wire [1:0] alu_op;

    // Main decoder 的模块，根据指令的操作码生成控制部件的控制信号
    main_decoder u_main_decoder (
        .instr(instr),
        .jump(jump),
        .branch(branch),
        .alu_src(alu_src),
        .mem_write(mem_write),
        .mem_to_reg(mem_to_reg),
        .reg_write(reg_write),
        .reg_dst(reg_dst),
        .alu_op(alu_op)
    );

    // ALU decoder 的模块，根据 alu_op 与 func 生成 alu_control 的信号

```

```

    alu_decoder u_alu_decoder (
        .func(func),
        .alu_op(alu_op),
        .alu_control(alu_control)
    );

endmodule

```

6.main_decoder.v

```

module main_decoder (
    input [5:0] instr,          // instruct[31:26] = 操作码

    //控制信号
    output reg jump,
    output reg branch,
    output reg alu_src,
    output reg mem_write,
    output reg mem_to_reg,
    output reg reg_write,
    output reg reg_dst,
    output reg [1:0] alu_op
);

always @(*) begin
    //默认值
    jump      = 1'b0;
    branch    = 1'b0;
    alu_src    = 1'b0;
    mem_write  = 1'b0;
    mem_to_reg = 1'b0;
    reg_write  = 1'b0;
    reg_dst    = 1'b0;
    alu_op     = 2'b00; //为 add 操作

    case(instr)
        //R 型指令
        6'b000000: begin
            reg_write = 1'b1;
            reg_dst   = 1'b1;
            alu_op     = 2'b10;
        end
        //lw
        6'b100011: begin
            reg_write = 1'b1;
            alu_src    = 1'b1;
            mem_to_reg = 1'b1;
        end
        //sw
        6'b101011: begin

```

```

        alu_src = 1'b1;
        mem_write = 1'b1;
    end
    //beq
    6'b000100: begin
        branch = 1'b1;
        alu_op = 2'b01;
    end
    //addi
    6'b001000: begin
        reg_write = 1'b1;
        alu_src = 1'b1;
    end
    //j
    6'b000010: begin
        jump = 1'b1;
    end
    default: begin
        //其他指令保持默认控制信号
    end
endcase
end
endmodule

```

7.alu_decoder.v

```

module alu_decoder (
    input [5:0] func,          // r 型指令的辅助操作码信号
    input [1:0] alu_op,        //来自 maindecoder
    output reg [2:0] alu_control //输出给 ALU 的控制信号
);

always @(*) begin
    case (alu_op)
        //lw/sw 指令
        2'b00: alu_control = 3'b010; //add
        //beq 指令
        2'b01: alu_control = 3'b110; //subtract

        //r 型指令, 根据 func 字段选择 alu 的操作
        2'b10: begin
            case (func)
                6'b100000: alu_control = 3'b010; // add
                6'b100010: alu_control = 3'b110; // sub
                6'b100100: alu_control = 3'b000; // and
                6'b100101: alu_control = 3'b001; // or
                6'b101010: alu_control = 3'b111; // slt
                default: alu_control = 3'b010; // 默认为 add
            endcase
        end
    end
end

```

```

        //默认为 add
        default: alu_control = 3'b010;    // add
    endcase
end
endmodule

```

8. datapath.v

```

module datapath(
    input wire      clk,
    input wire      rst,
    input wire      mem_to_reg,
    input wire      pc_src,
    input wire      alu_src,
    input wire      reg_dst,
    input wire      reg_write,
    input wire      jump,
    input wire [2:0] alu_control,
    output wire      overflow,
    output wire      zero,
    output wire [31:0] pc,
    input wire [31:0] instr,
    output wire [31:0] alu_out,
    output wire [31:0] write_data,
    input wire [31:0] read_data
);

    wire      inst_ce;
    wire [4:0] write_reg;
    wire [31:0] pc_next;
    wire [31:0] pc_next_branch;
    wire [31:0] pc_plus4;
    wire [31:0] pc_branch;
    wire [31:0] pc_jump;
    wire [31:0] sign_imm;
    wire [31:0] sign_imm_sl2;
    wire [31:0] src_a;
    wire [31:0] src_b;
    wire [31:0] result;
    assign inst_ce = 1'b1;
    assign pc_jump = {pc_plus4[31:28], instr[25:0], 2'b00};
    assign zero = (alu_out == 32'b0);
    assign overflow = 1'b0;
    // 程序计数器
    program_counter pc_inst (
        .clk      (clk),
        .rst      (rst),

```

```

        .inst_ce    (inst_ce),
        .pc_next    (pc_next),
        .pc_current (pc)
    );
    // pc + 4 的计算
    adder pc_plus4_adder (
        .a (pc),
        .b (32'd4),
        .y (pc_plus4)
    );
    // 立即数符号扩展
    signext sign_extend (
        .a (instr[15:0]),
        .y (sign_imm)
    );
    // 分支地址移位
    sl2 branch_shift (
        .a (sign_imm),
        .y (sign_imm_sl2)
    );
    // 计算分支目标地址
    adder branch_adder (
        .a (pc_plus4),
        .b (sign_imm_sl2),
        .y (pc_branch)
    );
    // 根据 pc_src 信号选择 pc 的下一个地址是 pc_plus4 还是 pc_branch
    mux2 #(.WIDTH(32)) pc_branch_mux (
        .a (pc_plus4),
        .b (pc_branch),
        .sel (pc_src),
        .y (pc_next_branch)
    );
    // 根据 jump 信号选择 pc 的下一个地址是 pc_next_branch 还是 pc_jump
    mux2 #(.WIDTH(32)) pc_jump_mux (
        .a (pc_next_branch),
        .b (pc_jump),
        .sel (jump),
        .y (pc_next)
    );
    // 寄存器堆
    regfile rf (
        .clk (clk),
        .we3 (reg_write),
        .ra1 (instr[25:21]),
        .ra2 (instr[20:16]),
        .wa3 (write_reg),
        .wd3 (result),
        .rd1 (src_a),
        .rd2 (write_data)
    );
    // 根据 reg_dst 信号选择写回寄存器的地址是 rt 还是 rd
    mux2 #(.WIDTH(5)) write_reg_mux (
        .a (instr[20:16]),

```

```

        .b (instr[15:11]),
        .sel (reg_dst),
        .y (write_reg)
    );
    // 根据 alu_src 信号选择 ALU 的第二个操作数是寄存器读出的数据还是立即数
    mux2 #(.WIDTH(32)) src_b_mux (
        .a (write_data),
        .b (sign_imm),
        .sel (alu_src),
        .y (src_b)
    );
    // ALU 单元
    alu alu_unit (
        .A (src_a),
        .B (src_b),
        .op (alu_control),
        .result (alu_out)
    );
    // 结果选择器，根据 mem_to_reg 信号选择写回寄存器的数据来源
    mux2 #(.WIDTH(32)) result_mux (
        .a (alu_out),
        .b (read_data),
        .sel (mem_to_reg),
        .y (result)
    );
endmodule

```

9. program_counter.v

```

module program_counter (
    input clk,
    input rst,
    input [31:0] pc_next,
    output reg [31:0] pc_current,
    input inst_ce //指令使能信号，控制 pc 是否更新
);

//处理 pc 地址更新，若有 rst 信号则把 pc_current 重置为 0
always @(posedge clk or posedge rst) begin
    if (rst) begin
        pc_current <= 32'h00000000;
    end else if (inst_ce) begin
        pc_current <= pc_next;
    end
end
endmodule

```

10.Adder.v

```
module adder(  
    input wire[31:0] a,b,  
    output wire[31:0] y  
);  
  
    //加法  
    assign y = a + b;  
endmodule
```

11.Signext.v

```
module signext(  
    input [15:0] a,  
    output [31:0] y);  
  
    //符号扩展  
    assign y = {{16{a[15]}}, a[15:0]};  
endmodule
```

12.sl2.v

```
module sl2(  
    input [31:0] a,  
    output [31:0] y);  
  
    //左移 2 位  
    assign y = {a[29:0], 2'b00};  
endmodule
```

13.mux2.v

```
module mux2 #(parameter WIDTH = 32)(  
    input wire [WIDTH-1:0] a,  
    input wire [WIDTH-1:0] b,  
    input wire sel,
```

```

    output wire [WIDTH-1:0] y
  );

  assign y = sel ? b : a;
  //调用方式: mux2 #(.WIDTH(32)) u_mux_name (
endmodule

```

14.regfile.v

```

module regfile(
    input wire clk,
    input wire we3,
    input wire[4:0] ra1,ra2,wa3,
    input wire[31:0] wd3,
    output wire[31:0] rd1,rd2
);

    reg [31:0] rf[31:0];
    always @(posedge clk) begin
        if(we3) begin
            rf[wa3] <= wd3;
        end
    end
    assign rd1 = (ra1 != 0) ? rf[ra1] : 0;
    assign rd2 = (ra2 != 0) ? rf[ra2] : 0;
endmodule

```

15.alu.v

```

module alu (
    input wire [31:0] A,
    input wire [31:0] B,
    input wire [2:0] op,
    output reg [31:0] result
);
    always @(*) begin
        case (op)
            3'b010: result = A + B;           // add
            3'b110: result = A - B;           // subtract
            3'b000: result = A & B;           // and
            3'b001: result = A | B;           // or
            3'b111: result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0; // slt
            default: result = 32'h00000000;
        endcase
    end
endmodule

```


--

备注：